Final Exam

COMP102: Data Structures with Python

University Mohammed VI Polytechnic — SASE
Duration: 3 hours — Total: **20 points**

Instructions. Answer all questions in the spaces provided. For single-choice MCQs, mark exactly one answer. For code questions, write clear Python code. Assume valid inputs. For traversal questions, follow the specified order exactly.

Name:

Question 1 — 2 pts

Each question is worth 0.25 points.

1. Python objects and aliasing. What is printed?

```
a = [1, 2]
b = a
a.append(3)
a = [9]
print(b)
```

- ○ A. `[1, 2]`
- ○ B. `[1, 2, 3]`
- ○ C. `[9]`
- ○ D. `NameError`

2. Which statement is correct?

- ○ A. Strings are mutable because characters can be accessed by index.
- ○ B. Assigning `a = b` copies a list.
- ○ C. Tuples can never contain mutable objects.
- ○ D. Lists can be changed in place; integers and strings are not changed in place.

3. Why is inserting after a known node in a singly linked list usually $O(1)$?

- ○ A. Nodes are stored contiguously.
- ○ B. Only a constant number of references changes.
- ○ C. The list is automatically sorted.
- ○ D. Python copies the whole list internally.

4. What is the preorder traversal?

```
        8
       / \
      3   10
     / \    \
    1   6    14
```

- ○ A. `1, 3, 6, 8, 10, 14`
- ○ B. `8, 3, 1, 6, 10, 14`
- ○ C. `1, 6, 3, 14, 10, 8`
- ○ D. `8, 10, 14, 3, 6, 1`

5. For a sparse graph with many vertices and relatively few edges, which representation is usually more memory-efficient?

- ○ A. Matrix
- ○ B. List
- ○ C. Heap
- ○ D. Sorted array

6. Dijkstra is appropriate when:

- ○ A. The graph is unweighted and only reachability matters.
- ○ B. Edge weights are non-negative and shortest paths are needed.
- ○ C. The graph has negative-weight edges.
- ○ D. The graph must be stored as a matrix.

7. An $O(n^2)$ algorithm takes 1 second for $n = 1000$. About how long for $n = 10000$, with the same constants?

- ○ A. 10 s
- ○ B. 20 s
- ○ C. 100 s
- ○ D. 10000 s

8. What is the postorder traversal of the BST in Q4?

- ○ A. `1, 6, 3, 14, 10, 8`
- ○ B. `8, 3, 1, 6, 10, 14`
- ○ C. `1, 3, 6, 8, 10, 14`
- ○ D. `14, 10, 8, 6, 3, 1`

Question 2 — 0.5 pts

A company database uses the following three tables:

```
Employee(id, name, department)

Project(code, title, budget)

Assignment(employee_id, project_code, hours)
```

Write one SQL query that returns:

- each employee's name,
- each project title,
- the number of hours assigned to that employee on that project.

Question 3 **0.5 pts**

Explain when a document database is a better fit than a normalized SQL schema. Give one example.

Question 4 **0.5 pts**

Explain the difference between an abstract data type and a data structure. Give one example.

Question 5 **0.5 pts**

A recursive algorithm solves a problem of size n by splitting it into **three** subproblems of size $n/2$, then spending $O(n)$ extra work to combine the results.
Write the recurrence in the form $T(n) = aT(n/b) + f(n)$, identify a, b, and $f(n)$, then give the final asymptotic complexity. Justify your answer.

Question 6 **1 pt**

Simulate merge sort on [9, 4, 7, 1, 6, 3, 8, 2]. Show the recursive splitting until sublists of size 1, then show the merge steps until the final sorted list.

Question 7 **2 pts**

Starting from an empty AVL tree, insert the following values in order:

1, 2, 3, 4, 5, 7, 6

Draw the tree after each insertion. When a rotation is needed, draw the tree **before** and **after** the rotation. You do not need to compute heights or balance factors.

Question 8 **2 pts**

Use a hash table with $M = 4$ buckets and $h(k) = k \bmod 4$. Each bucket stores a list of key-value pairs. Insertions append to the end of the bucket if the key is new. Deletions remove the matching pair; if the key is absent, do nothing.

```
put(12, "A")
put(7,  "B")
put(3,  "C")
put(8,  "D")
```

```
put(13, "E")
delete(7,  "B")
put(18, "F")
delete(3, "G")
```

Show the hash table after each step:

Question 9 **2 pts**

Start from an empty **min-heap**. Insert 12, 7, 15, 3, 10, 8 in order. Draw the heap as a **tree** after each insertion. Then remove the root once. Draw the tree after popping the root.

Question 10 **3 pts**

Ten students participate in different clubs. The students are $A, B, C, D, E, F, G, H, I, J$.
Their club memberships are:

- A: Chess, Music
- B: Chess, Robotics
- C: Music, Art
- D: Robotics, Debate
- E: Art, Debate
- F: Music, Debate
- G: Tennis
- H: Tennis, Swimming
- I: Swimming
- J: Photography

Two students are connected by an undirected edge if they share at least one club.
Use alphabetical order when several neighbors are available. For iterative DFS, when several unvisited neighbors are available, visit the alphabetically first neighbor next.

- (a) Draw the graph.
- (b) Find the adjacency list.
- (c) Find the adjacency matrix using order $A, B, C, D, E, F, G, H, I, J$.
- (d) Run a BFS from A and output the BFS order.
- (e) Run an iterative DFS from A and output the DFS order.
- (f) Find the number of connected components.

Question 11 **2 pts**

Implement a queue class using `collections.deque`. You should implement the following methods:

- `enqueue(value)`
- `dequeue()`
- `front()`
- `is_empty()`
- `size()`

Question 12 **1 pt**

Implement `choose(n, k)` recursively. Use $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.

Question 13 **1 pt**

Assume a binary tree node has attributes `value`, `left`, and `right`. A leaf has no children. Implement:

```
def count_leaves(root):
    pass
```

Return `0` for an empty tree.

Question 14 **1 pt**

Implement the function below. Inputs `a` and `b` are sorted ascending. Return a new sorted list containing all values from both lists. Do not use `sort()` or `sorted()`.

```
def merge_sorted(a, b):
    pass
```

Question 15 **1 pt**

Implement the function below. Return `True` if two distinct elements in `values` sum to `target`; otherwise return `False`. A correct $O(n^2)$ solution receives up to **0.25 pts**; a correct better-than-quadratic solution receives full credit.

```
def two_sum(values, target):
    pass
```